

Building an end-to-end real-time data engineering project using Azure Data Factory, Azure Databricks, Azure Data Lake Storage, and Power BI involves several steps. Below is a detailed guide to help you set up this architecture and implement a CI/CD pipeline.

High-Level Architecture

1. Data Ingestion:

- Use Azure Data Factory (ADF) for orchestrating data ingestion from various sources.

2. Data Processing:

- Use Azure Databricks for data transformation and processing.

3. Data Storage:

- Store raw and processed data in Azure Data Lake Storage (ADLS).

4. Data Visualization:

- Use Power BI to create interactive dashboards and reports.

Technologies Used

- Infrastructure: Azure Resource Manager (ARM) templates, Terraform
- Source Control: GitHub or Azure Repos
- CI/CD: Azure Pipelines or GitHub Actions
- Data Orchestration: Azure Data Factory
- Data Processing: Azure Databricks

- Data Storage: Azure Data Lake Storage
- Visualization: Power BI

Step-by-Step Implementation

Step 1: Set Up Infrastructure

1. Provision Resources using Terraform

Create a Terraform script to provision necessary resources like Azure Data Lake Storage, Azure Databricks workspace, and Azure Data Factory.

```
provider "azurerm" {  
  
  features {}  
  
}
```

```
resource "azurerm_resource_group" "rg" {  
  
  name     = "rg-mydataengineering"  
  
  location = "East US"  
  
}
```

```
resource "azurerm_storage_account" "storage" {  
  
  name                = "mystorageaccount"  
  
  resource_group_name = azurerm_resource_group.rg.name  
  
  location            = azurerm_resource_group.rg.location  
  
  account_tier        = "Standard"
```

```
account_replication_type = "LRS"
}

resource "azurerm_data_factory" "adf" {
  name          = "mydatafactory"
  location      = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_databricks_workspace" "databricks" {
  name          = "mydatabricks"
  location      = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  sku           = "standard"
}
```

Add more resources as needed

Deploy with:

```
terraform init
```

```
terraform apply
```

Step 2: Set Up Data Ingestion

1. Create Azure Data Factory Pipeline

Use Azure Data Factory to create pipelines that ingest data from various sources (e.g., Azure Blob Storage, REST APIs, on-premises databases) and load it into ADLS.

```
{
  "name": "IngestDataPipeline",
  "properties": {
    "activities": [
      {
        "name": "CopyFromBlobToADLS",
        "type": "Copy",
        "inputs": [
          {
            "name": "BlobSource"
          }
        ],
        "outputs": [
          {
            "name": "ADLSDestination"
          }
        ],
        "typeProperties": {
          "source": {
            "type": "AzureBlobStorage"
          },
          "sink": {
```

```

        "type": "AzureDataLakeStore"
    }
}
}
]
}
}

```

Step 3: Set Up Data Processing

1. Create Databricks Notebooks

Develop Databricks notebooks to process and transform the ingested data.

Databricks notebook source

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("DataProcessing").getOrCreate()
```

Read data from ADLS

```
df = spark.read.format("csv").option("header",
"true").load("adl://mystorageaccount.azuredatalakestore.net/data/raw/")
```

Data transformations

```
df_transformed = df.withColumn("new_column", df["existing_column"] * 2)
```

```
# Write transformed data back to ADLS
```

```
df_transformed.write.format("parquet").save("adl://mystorageaccount.azuredatalakestore.net/data/processed/")
```

Step 4: Set Up CI/CD Pipeline

1. Azure Pipelines Configuration

trigger:

branches:

include:

- main

variables:

- name: azureSubscription

value: 'your-azure-subscription-id'

stages:

- stage: DeployInfrastructure

jobs:

- job: Terraform

pool:

vmImage: 'ubuntu-latest'

steps:

- task: AzureCLI@2

inputs:

azureSubscription: \$(azureSubscription)

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

terraform init

terraform apply -auto-approve

- stage: IngestData

dependsOn: DeployInfrastructure

jobs:

- job: ADFPipeline

pool:

vmImage: 'ubuntu-latest'

steps:

- task: AzureCLI@2

inputs:

azureSubscription: \$(azureSubscription)

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

az datafactory pipeline create --resource-group \$(resourceGroup) --factory-name
\$(dataFactoryName) --name "IngestDataPipeline" --pipeline-file @path/to/pipeline.json

- stage: ProcessData

dependsOn: IngestData

jobs:

- job: Databricks

pool:

vmImage: 'ubuntu-latest'

steps:

- task: DatabricksSubmitRun@1

inputs:

azureSubscription: \$(azureSubscription)

workspaceUrl: 'https://<databricks-instance>#'

existingClusterId: 'cluster-id'

notebookPath: '/Workspace/Path/To/Notebook'

libraries: '["jar": "dbfs:/path/to/library.jar"]'

- stage: VisualizeData

dependsOn: ProcessData

jobs:

- job: PowerBI

pool:

vmImage: 'windows-latest'

steps:

- task: PowerBIDeploy@1

inputs:

workspaceId: '<PowerBI-Workspace-ID>'

datasetId: '<Dataset-ID>'

accessKey: '<PowerBI-Access-Key>'

pbixPath: 'path/to/your-report.pbix'

Step 5: Set Up Visualization

1. Create Power BI Dashboard

Configure Power BI to read data from the processed dataset in ADLS or Azure SQL Database. Create interactive dashboards and reports based on the processed data.

let

```
Source = AzureDataLakeStorage.Contents("https://mystorageaccount.dfs.core.windows.net/"),
```

```
ProcessedData = Source{[Name="data/processed"]}[Content]
```

in

```
ProcessedData
```

Summary

This setup provides a comprehensive end-to-end real-time data engineering solution using Azure Data Factory, Azure Databricks, Azure Data Lake Storage, and Power BI. It involves:

1. Infrastructure Provisioning: Terraform scripts for automated infrastructure setup.
2. Data Ingestion: Azure Data Factory for orchestrating data ingestion.
3. Data Processing: Azure Databricks for data transformation and processing.
4. CI/CD Pipeline: Azure Pipelines for continuous integration and deployment.
5. Data Visualization: Power BI for creating interactive dashboards and reports.

This framework can be adapted and extended based on specific requirements, technologies, and cloud providers.